CSE-574: Cloud Computing

CloudBurst

Deadline-Aware, Duplication-Enabled, HEFT-Seeded PSO for Cost and Energy Efficient Scientific Workflow Scheduling

[Name] [Roll Number] [Branch]

The Problem and the Research Papers

The problem it solves

- Scientific workflows (Montage, CyberShake, Epigenomics, LIGO) are DAGs of dependent tasks.
- Clouds offer heterogeneous VMs, billed per time period, with boot delays.
- Goal: map tasks to VMs so the deadline is met while cost and energy are minimized.
- Poor schedules waste money (partly used billing hours), energy and time.

Paper	What we take from it
[1] Task duplication (HEFT-TD)	Duplicate parent tasks to cut data transfer and idle time
[2] PSO provisioning	PSO encoding, hourly billing, boot time, Deb's deadline handling
[3] CEAS	Deadline split into sub-makespans, energy awareness, slack use

Papers: [1] Genez et al. 2018; [2] Rodriguez and Buyya 2014; [3] Li et al. 2018

Paper 1: Task Duplication (Genez et al., 2018)

What it does

- Targets idle VMs caused by data transfers between dependent tasks.
- Extends HEFT and Lookahead into HEFT-TD and Lookahead-TD.

Reported results

- About 9% (HEFT-TD) and 12% (Lookahead-TD) lower makespan on average.

Key idea

- Temporarily duplicate the critical parent on each candidate VM; keep the copy only if finish time improves.
- Rescheduling removes originals fully served by duplicates.

Gaps still present

- No deadline or budget; cost rose +10% to +45% for some workflows.
- Uneven gains; about 10 to 100 times slower.
- Simplified model: no boot time or hourly billing.

How CloudBurst addresses it

- Duplication decisions also consider cost, energy and the deadline.

[1] Genez et al., Scheduling Scientific Workflows on Clouds Using a Task Duplication Approach, IEEE/ACM UCC 2018

Paper 2: PSO Provisioning (Rodriguez and Buyya, 2014)

What it does

- Minimizes cost under a deadline: which VMs to lease, when, and where each task runs.

Reported results

- Meets deadlines far more reliably than IC-PCP; cheaper than SCS where both meet the deadline.

Key idea

- One dimension per task; integer part = VM index.
- Fitness = cost under hourly billing; Deb's rules enforce the deadline.
- Models boot time (97 s) and variability.

Gaps still present

- Slow, variable between runs, sensitive to initial VM pool.
- No energy awareness, no duplication.
- Static schedule; only two baselines.

How CloudBurst addresses it

- HEFT seeding gives a strong start; energy and duplication are added; Deb's rules are kept.

[2] Rodriguez and Buyya, Deadline Based Resource Provisioning and Scheduling Algorithm for Scientific Workflows on Clouds, IEEE TCC 2014

Paper 3: CEAS (Li et al., 2018) and the Combined Gap

What it does

- Minimizes cost, then energy, under a deadline via five chained polynomial-time steps.

Reported results

- Lowest cost vs HEFT and MOHEFT; lowest energy vs EHEFT and EES (plots only).

Key idea

- Deadline split into per-task sub-makespans; VM chosen by cost utility.
- Task merging, VM reuse, then DVFS slack reclamation.

Gaps still present

- Greedy; no optimality guarantee; no duplication.
- Assumes DVFS control, rare on public IaaS.
- No boot time or variability; dated baselines.

Combined research gap: no approach combines task duplication, deadline-aware cost minimization under billing periods, and energy awareness in a global search. [1] ignores deadline and cost, [2] ignores energy and duplication, [3] is greedy with no duplication.

[3] Li et al., Cost and Energy Aware Scheduling Algorithm for Scientific Workflows with Deadline Constraint in Clouds, IEEE TSC 2018

HEFT: Heterogeneous Earliest Finish Time

A fast list-scheduling heuristic: rank tasks by urgency, then place each on the VM that finishes it earliest.

1. Prioritize tasks by upward rank › 2. Assign in rank order to the VM with earliest finish time

Formula	Meaning
rank_u(t_i) = w_i + max_j (c_ij + rank_u(t_j))	Upward rank: average execution time plus longest path to exit, incl. communication
w_i = L_i / mean(s_k)	Average execution time over VM types (L in MI, s in MIPS)
EST(t_i,k) = max(avail(k), max_j [AFT(t_j) + c_ji])	Earliest start: VM free and parent data arrived
EFT(t_i,k) = EST(t_i,k) + L_i / s_k	Earliest finish time of task i on VM k
Assign t_i to argmin_k EFT(t_i,k)	Pick the VM with the earliest finish

Limitation: optimizes only makespan; ignores cost, energy and deadline; greedy.

[1] builds on HEFT; CloudBurst uses it as seed and baseline

PSO: Particle Swarm Optimization

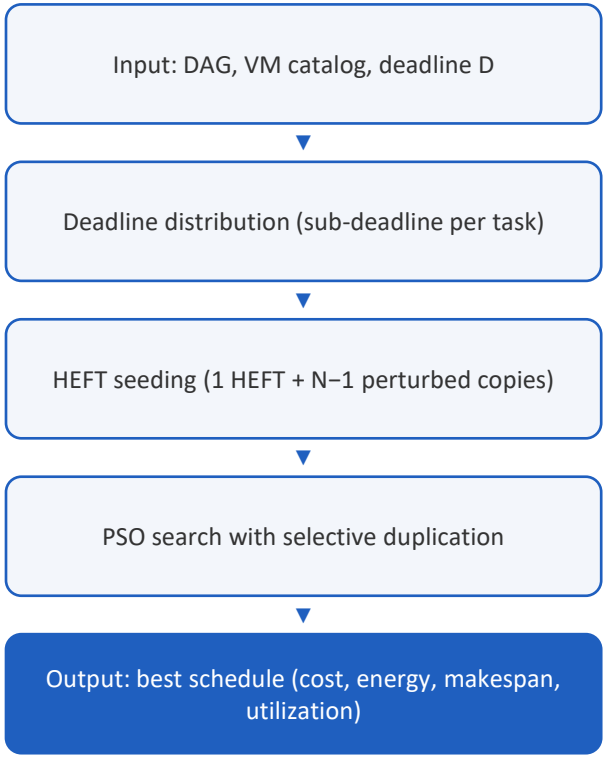
A swarm of candidate solutions moves through the search space, pulled toward its own best and the swarm's best. In scheduling, a particle is a full task-to-VM mapping.

Formula	Meaning
$v_i(t+1) = w \cdot v_i(t) + c1 \cdot r1 \cdot (pbest_i - x_i) + c2 \cdot r2 \cdot (gbest - x_i)$	Momentum plus pull toward personal and global best (r1, r2 in [0,1])
$x_i(t+1) = x_i(t) + v_i(t+1)$	Position update
vm(task d) = floor(x_{id}), clamped to valid indices	Integer part of the coordinate is the VM index, as in [2]
$w(t) = w_{max} - (w_{max} - w_{min}) \cdot t / I$	Decreasing inertia: explore early, refine late

Limitation: slow and variable between runs; wastes iterations when starting from random solutions.

[2] uses PSO for workflow scheduling

Our Algorithm: Architecture and Pseudocode




```
Algorithm: CloudBurst (DD-HPSO)
1. Compute upward ranks; build HEFT schedule (M, C, E)
2. Distribute deadline: SD_i = AFT_HEFT(i)·D / M_HEFT
3. Init swarm: 1 HEFT particle, N−1 perturbed copies
4. for iter = 1 to I:
5.   for each particle p:
6.     decode p to schedule (rank order, boot time)
7.     SelectiveDuplication(schedule)
8.     compute C, E, M, U; evaluate F(p), V(p)
9.     update pbest, gbest using Deb's rules
10.  update velocity, position (adaptive inertia)
11. return schedule of gbest
```

Selective duplication rule

- For each task in topological order, for each critical parent on another VM:
- duplicate it onto VM(t_i) if an idle slot fits, AFT(t_i) drops, and extra cost/energy is under a threshold (prefer already-paid billing time).

Our Algorithm: Mathematical Modelling

Workflow G=(T,E), task length L_i (MI), edge data d_ij; VM type k: speed s_k, price p_k per period τ, bandwidth B, boot T_boot, power P_busy, P_idle.

Formula	Explanation
ET(i,k) = L_i / s_k	Execution time
TT(i,j) = d_ij / B (0 on same VM)	Data transfer time
AFT(i) = EST(i) + ET(i,k)	Finish time; VM available only after T_boot
M = AFT(t_exit)	Makespan
C = Σ ceil((T_end − T_start)/τ)·p_k	Cost: leased time (incl. boot) rounded up to billing periods
E = Σ (P_busy·T_busy + P_idle·T_idle)	Energy: busy and idle power; no DVFS needed
U = busy time / leased time	Resource utilization
SD_i = AFT_HEFT(i)·D / M_HEFT	Sub-deadline (HEFT-based variant of [3]'s idea)

Objective

```
min F = w1·C/C_ref + w2·E/E_ref +
w3·M/M_ref
s.t. M ≤ D, precedence, w1+w2+w3=1
```

Violation V = max(0, M − D).
Deb's rules [2]: feasible beats infeasible; both feasible: smaller F; both infeasible: smaller V.
Refs from HEFT, so F = 1.0 means "same as HEFT".

How Our Algorithm Is Better

Feature	HEFT / Min-Min / Max-Min	[1] HEFT-TD	[2] PSO	[3] CEAS	CloudBurst
Deadline-aware	No	No	Yes	Yes	Yes
Cost-aware	No	No (incidental)	Yes	Yes	Yes
Energy-aware	No	No	No	Yes (DVFS)	Yes (busy/idle)
Task duplication	No	Yes (finish-time)	No	No	Yes (cost, deadline aware)
Global search	No	No	Yes	No	Yes
Boot time and billing	No	No	Yes	Billing only	Yes
Heuristic-seeded start	n/a	n/a	No	n/a	Yes

Advantage 1

HEFT seeding should reduce PSO slowness and run-to-run variability [2] (expected).

Advantage 2

Duplication accepted only if cost, energy and deadline allow, addressing cost rises in [1].

Advantage 3

One objective balances cost, energy, makespan; no DVFS control needed, unlike [3].

Expected benefits, to be verified through simulation. Trade-off: longer scheduling time than pure heuristics.

Implementation Overview

Workflow input › Algorithm implementation › Simulation › **Comparison and analysis**

Simulator

WorkflowSim (built on CloudSim)

Workflows

Pegasus: Montage, CyberShake, Epigenomics, LIGO; varying sizes

Compared against

HEFT, Min-Min, Max-Min, plain PSO, algorithms from [1]-[3] where feasible, recent (2025/2026) papers

Metrics

Makespan, cost, energy, utilization, deadline success rate

Next: implementation and results for the 2nd evaluation.